

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3623 - DATABASE PROGRAMMING 2025/2026 - SEM 1

Lab Assignment 1 (5%) - Indexes

Name	:	DHESHIEGHAN A/L SARAVANA MOORTHY
Matric Id	:	A23CS0072

Name	:	PRAVINRAJ A/L SIVABATHI
Matric Id	:	A23CS0171

Name	:	WOO CHENG SHUAN
Matric Id	:	A23CS0283

1. WHAT IS A UNIQUE INDEX?

A **Unique Index** is a database feature that ensures all the values in a specific column (or group of columns) are **unique**, meaning no duplicate values are allowed. It helps maintain **data integrity** by preventing multiple rows from having the same value in the indexed column.

EXAMPLE :

```
ALTER TABLE customer  
ADD UNIQUE idx_email(email);
```

This command adds a **unique constraint** on the `email` column so that no two customers can have the same email address.

2. HOW IT DIFFERS FROM PRIMARY KEY

Feature	Primary Key	Unique Index
Purpose	Identifies each record in a table uniquely.	Ensures that values in a specific column (or columns) are unique.
Number per Table	Only one primary key per table.	You can have multiple unique indexes in a table.
NULL Values	Cannot contain NULL values.	Can contain NULL values (since NULL is not treated as a value).
Automatic Index	Automatically creates a unique index on the primary key column.	Must be manually created using UNIQUE keyword.
Main Role	Used to uniquely identify each row.	Used to prevent duplicate values in certain columns.

3. ADVANTAGES & DISADVANTAGES

Aspect	Advantages	Disadvantages
Data Integrity	Ensures no duplicate values are inserted in the column.	-
Data Accuracy	Maintains consistent and reliable data in the table.	-
Performance	Speeds up searches and lookups when querying by the unique column.	May slow down INSERT or UPDATE operations because the database must check for duplicates first.
Storage Space	-	Uses extra disk space to store index information.
Flexibility	Can be applied to multiple columns, allowing flexible uniqueness rules.	-

4. CREATE A TABLE

```

1 • CREATE DATABASE index_lab;
2 • USE index_lab;
3
Output
Action Output
# Time Action
✓ 1 00:51:40 CREATE DATABASE index_lab

```

```

4 #step3
5 • CREATE TABLE customer (
6     cust_id INT AUTO_INCREMENT,
7     email VARCHAR(50),
8     phone VARCHAR(15),
9     PRIMARY KEY (cust_id)
10 );

```

✓ 3 01:19:39 CREATE TABLE customer (cust_id INT AUTO_INCREM... 0 row(s) affected

In this step, a new database named **index_lab** is created and selected for use. Then, a table called **customer** is created with three columns: **cust_id**, **email**, and **phone**. The **cust_id** column is set as the **primary key** with **AUTO_INCREMENT**, meaning it will automatically assign a unique ID to each customer. This table serves as the basic structure for storing customer information before adding any indexes or constraints.

4. INSERT 10 RECORDS

```
12  #step4
13  •  INSERT INTO customer (email, phone)
14  VALUES
15  ('woo04@gmail.com', '0112223333'),
16  ('cheng02@gmail.com', '0112224444'),
17  ('shuan21@gmail.com', '0112225555'),
18  ('jayne36@gmail.com', '0112226666'),
19  ('shawn14@gmail.com', '0112227777'),
20  ('jesicca45@gmail.com', '0112228888'),
21  ('kate67@gmail.com', '0112229999'),
22  ('jacky98@gmail.com', '0114443333'),
23  ('nic76@gmail.com', '0115553333'),
24  ('woo04@gmail.com', '0116663333')
```

In this step, I inserted 10 records into the **customer** table using the INSERT INTO SQL statement. Each record contains an email and phone number for different customers. This process helps to populate the database with sample data that can be used for testing and further operations such as retrieving, updating, or deleting records. The query executed successfully, showing that 10 rows were added with no duplicates or warnings.

✓	4	01:19:45	INSERT INTO customer (email, phone) VALUES ('woo04...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0
---	---	----------	---	--

5. ADD A UNIQUE INDEX

```
26 ✖ ALTER TABLE customer
27 ADD UNIQUE idx_customer (email);
```

✖ 5 01:20:24 ALTER TABLE customer ADD UNIQUE idx_customer (e... Error Code: 1062. Duplicate entry 'woo04@gmail.com' for ...

In this step, I tried imposing some data integrity on the customer table, by creating a UNIQUE index on the email column. This was done to ensure nobody is using the same email twice, otherwise in future there will be copies. But unfortunately the above query didn't work and it returns with Error Code 1062 as the table already has duplicate email address, in this case 'woo04@gmail.com' is found multiple times. You need to find and auto-correct / delete the duplicates, before being able to add this unique index.

```
26 ✖ DELETE FROM customer
27 WHERE cust_id = 10;
28
29 • ALTER TABLE customer
30 ADD UNIQUE idx_customer (email);
```

```
✓ 7 01:28:49 DELETE FROM customer WHERE cust_id = 10
✓ 8 01:28:56 ALTER TABLE customer ADD UNIQUE idx_customer (email)
```

In this step, I first **deleted** the record with `cust_id = 10` from the customer table. This deletion likely **removed a duplicate email entry** that was causing the previous operation to fail. Following the deletion, I successfully executed the **ALTER TABLE** statement to **add a UNIQUE index** (`idx_customer`) to the **email** column. This ensures all future entries into the customer table will have a unique email address, enforcing **data integrity**.

6. INSERT DUPLICATE EMAIL AGAIN

```
32      #step6
33      •  INSERT INTO customer (email, phone)
34      VALUES
35      ('woo04@gmail.com', '0112223333')
```

✖ 9 01:31:47 INSERT INTO customer (email, phone) VALUES (woo... Error Code: 1062. Duplicate entry 'woo04@gmail.com'f...

This step was a **test** designed to show that the **UNIQUE index** added to the email column in the preceding steps is now **working as intended**. When I attempted to insert the email 'woo04@gmail.com' (a value already present in the table), the database system immediately **prevented the insertion** and returned **Error Code 1062** (Duplicate entry). This confirms that the **data integrity** rule requiring unique customer emails is successfully enforced by the database.

7. USE 'SHOW INDEX FROM customer'

37 #step7

38  SHOW INDEX FROM customer;

Result Grid Filter Rows: Export: Wrap Cell Content:											
	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type
▶	customer	0	PRIMARY	1	cust_id	A	9	NULL	NULL		BTREE
	customer	0	idx_customer	1	email	A	9	NULL	NULL	YES	BTREE

To resolve the duplicate email issue, I first executed a **DELETE** statement to remove the problematic record (`cust_id = 10`). This cleanup allowed the subsequent **ALTER TABLE** command to successfully add the **UNIQUE index** (`idx_customer`) to the `email` column, enforcing a rule that all future emails must be distinct.

To test this new constraint, I attempted to **INSERT** a record with a known duplicate email (`'woo04@gmail.com'`). As intended, the database **blocked the operation** and returned **Error Code 1062**, confirming the constraint was active.

Finally, the **SHOW INDEX FROM customer;** command was used to verify the table structure. The output confirmed the existence of both the **PRIMARY** key and the new **idx_customer** (on the `email` column), both reporting a **non_unique** value of **0**. This final step confirmed that data integrity is now actively secured by the database.

8. HOW A UNIQUE INDEX IMPROVES LOOKUP PERFORMANCE?

A unique index dramatically improves lookup performance by leveraging a highly efficient data structure, typically a **B-Tree (Balanced Tree)**. Instead of requiring the database to perform a slow **full table scan** (reading every single row) to find a record, the B-Tree organizes the indexed column's data (like email addresses) in a sorted, hierarchical structure.

When a search is performed, the database quickly **traverses the B-Tree** from the root down to the leaf nodes. This **logarithmic time complexity $O(\log n)$** ensures that the time required for a lookup remains minimal, even as the table grows to millions of records. Furthermore, the index's **unique constraint** is key: once the database finds the single matching entry (e.g., a specific email), it immediately **stops searching**, retrieves the pointer to the full row's data, and fetches the complete record. This combination of a highly organized structure and a guaranteed stopping condition makes indexed lookups significantly faster than unindexed lookups.

9. COMPARE QUERY PERFORMANCE

WITH INDEX

```
43 #with index
44 • EXPLAIN SELECT * FROM customer
45 WHERE email = 'woo04@gmail.com';
```

Result Grid Filter Rows: Export: Wrap Cell Content:												
id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra	
1	SIMPLE	customer	NULL	const	idx_customer	idx_customer	203	const	1	100.00	NULL	

WITHOUT INDEX

```
46 #without index
47 • EXPLAIN SELECT * FROM customer_no_index
48 WHERE email = 'woo04@gmail.com';
```

Result Grid Filter Rows: Export: Wrap Cell Content:												
id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra	
1	SIMPLE	customer_no_index	NULL	ALL	NULL	NULL	NULL	NULL	9	11.11	Using where	

Feature	Execution WITH Index (customer table)	Execution WITHOUT Index (customer_no_index table)
EXPLAIN Type	const (Constant)	ALL (Full Table Scan)
Key Used	idx_customer (The Unique Index)	NULL (No index used)
possible_keys	idx_customer	NULL (No index available)
Rows Examined	1	9
Extra Information	NULL (Clean, direct lookup)	Using where (Had to check the WHERE condition on every row read)
Performance	Extremely Fast (Direct lookup via B-Tree)	Slow (Sequential check of every row)
Scalability	Excellent (O(log n)) – Time barely increases with table size.	Poor (O(n)) – Time increases linearly with table size.

10. REFLECT: WHEN IS A UNIQUE INDEX NOT APPROPRIATE FOR A COLUMN?

1. Expected Duplicate Data

If the column is designed to store values that are naturally repeated, a unique index will cause insert and update operations to fail (Error 1062).

Examples:

- **first_name** or **last_name**: Many customers will share the same first or last name.
- **status** (e.g., 'Active', 'Inactive', 'Pending'): There are only a few possible values that repeat across all rows.
- **city** or **zip_code**: Many different customers live in the same location.
- **order_date**: Multiple orders can be placed on the same day.

2. High Write/Update Load (Performance Overhead)

While indexes speed up **reads** (SELECT), they **slow down writes** (INSERT, UPDATE, DELETE).

- 1) Every time a row is inserted or updated, the database must perform extra work to:
- 2) **Check for Uniqueness**: Read the index to ensure the new value does not already exist.
- 3) **Maintain the Index**: Update the B-Tree structure to include the new value (or remove/update the old value).
- 4) If a table has a very high volume of insertions or updates, the cumulative time spent maintaining many indexes (especially unique ones) can slow down the overall application more than the indexing speeds up reads.

3. Extremely Low Cardinality

Cardinality refers to the number of unique values in a column. If a column has very few unique values relative to the total number of rows (low cardinality), an index is generally not helpful for performance.

- 1) **Example**: A boolean `is_active` column (only two possible values: True or False). If you search for all 'True' rows, the database might find it faster to perform a **full table scan** than to traverse the index, as the index would point to almost half the table's rows anyway.

THANK YOU



univteknologimalaysia



utm.my



utmofficial